



UNIVERSIDAD DE BURGOS

APLICACIONES DE BASES DE DATOS

GESTION DE CATERING

ANALISIS SQL GENERADAS POR HIBERNATE

Docente: Pablo García Díaz

Luis Menéndez Ramos

lmr1027@alu.ubu.es

Contenido

1	Introducción	2
2	Configuración y captura de SQL	3
3	Análisis de operaciones SQL	4
3.1	Insertar compra correcta	4
3.2	Compra con cliente inexistente	6
3.3	Compra con menú inexistente.....	6
3.4	Insertar compra con fecha y cliente existentes	7
3.5	Quitar descuento del cliente	7
3.6	Información completa con grafos de entidades	10
4	Reflexión personal.....	13
5	Conclusión	13

1 Introducción

El objetivo de este documento es analizar las operaciones SQL generadas por Hibernate al ejecutar las transacciones implementadas en un sistema de gestión de catering basado en JPA. Para ello, se han ejecutado los test proporcionado en la clase *TestClient*, capturando las consultas SQL producidas de forma automática por el framework en cada caso.

A partir de estas trazas, se estudian las distintas consultas generadas, justificándola en la función lógica de negocio implementada.

2 Configuración y captura de SQL

Para la obtención de las consultas SQL generadas por Hibernate, se ha configurado el fichero *persistence.xml* localizado en el directorio *META-INF* dentro de *src/main/java*. En dicho fichero, se han modificado las propiedades *hibernate.show_sql* y *hibernate.format_sql*, cambiando su valor de false a true, con el objetivo de visualizar las sentencias SQL generadas y facilitar su lectura.

Properties

This table lists all properties that are defined for this persistence unit.

Name	Value
hibernate.dialect	org.hibernate.dialect.Oracle10gDialect
hibernate.connection.driver_class	oracle.jdbc.driver.OracleDriver
hibernate.connection.url	jdbc:oracle:thin:@localhost:1521:XE
hibernate.default_schema	HR
hibernate.connection.username	HR
hibernate.connection.password	hr
hibernate.connection.autocommit	false
hibernate.show_sql	true
hibernate.format_sql	true
hibernate.generate_statistics	false
hibernate.use_sql_comments	false
hibernate.connection.pool_size	5

Una vez realizada la configuración, se ha ejecutado los test de la clase *TestClient*, capturando las trazas SQL generadas por Hibernate en cada transacción.

3 Análisis de operaciones SQL

En este apartado se realiza el análisis de consultas SQL generadas por Hibernate durante la ejecución de las distintas transacciones implementadas.

3.1 Insertar compra correcta

En primer lugar, se genera la siguiente consulta:

Hibernate:

```
select
    cliente0_.cif as cif1_1_0_,
    cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
    cliente0_.descripcion as descripcion2_1_0_,
    cliente0_.ciudad as ciudad3_1_0_,
    cliente0_.cp as cp4_1_0_,
    cliente0_.direccion as direccion5_1_0_,
    bonoclient1_.idbonocliente as idbonocliente1_0_1_,
    bonoclient1_.bono as bono2_0_1_,
    bonoclient1_.descuento as descuento3_0_1_
from
    HR.Cliente cliente0_
left outer join
    HR.BonoCliente bonoclient1_
        on cliente0_.IDBONOCLIENTE=bonoclient1_.idbonocliente
where
    cliente0_.cif=?
```

Esta consulta recupera el cliente junto con su bono mediante un LEFT OUTER JOIN, lo que permite obtener el descuento asociado necesario para el cálculo del importe.

Luego, se realiza la consulta sobre menú:

Hibernate:

```
select
    menu0_.idmenu as idmenu1_3_0_,
    menu0_.descripcion as descripcion2_3_0_,
    menu0_.precio as precio3_3_0_
from
    HR.Menu menu0_
where
    menu0_.idmenu=?
```

Con esta consulta obtenemos el precio del menú, dato necesario para poder calcular el importe total de la compra.

Después, se ejecuta la siguiente consulta

Hibernate:

```
select
    compra0_.cif as cif1_2_0_,
    compra0_.fecha as fecha2_2_0_,
    compra0_.importe as importe3_2_0_,
    compra0_.IDMENU as idmenu5_2_0_,
    compra0_.personas as personas4_2_0_,
    cliente1_.cif as cif1_1_1_,
    cliente1_.IDBONOCLIENTE as idbonocliente6_1_1_,
    cliente1_.descripcion as descripcion2_1_1_,
    cliente1_.ciudad as ciudad3_1_1_,
    cliente1_.cp as cp4_1_1_,
    cliente1_.direccion as direccion5_1_1_,
    bonoclient2_.idbonocliente as idbonocliente1_0_2_,
    bonoclient2_.bono as bono2_0_2_,
    bonoclient2_.descuento as descuento3_0_2_,
    menu3_.idmenu as idmenu1_3_3_,
    menu3_.descripcion as descripcion2_3_3_,
    menu3_.precio as precio3_3_3_
from
    HR.Compra compra0_
left outer join
    HR.Cliente cliente1_
        on compra0_.CIF=cliente1_.cif
left outer join
    HR.BonoCliente bonoclient2_
        on cliente1_.IDBONOCLIENTE=bonoclient2_.idbonocliente
left outer join
    HR.Menu menu3_
        on compra0_.IDMENU=menu3_.idmenu
where
    compra0_.cif=? and compra0_.fecha=?
```

Con esta consulta comprobamos si existe una compra con el mismo CIF y fecha, con esto logramos evitar que se duplique antes de realizar la inserción.

Y finalmente realizamos la inserción:

Hibernate:

```
insert
into
    HR.Compra
    (importe, IDMENU, personas, cif, fecha)
values
    (?, ?, ?, ?, ?)
```

En esta operación se almacena la nueva compra con el importe que se había calculado previamente. Este proceso implica 3 consultas select y 1 insert por

cada compra, garantizando la validación de datos, pero aumenta el numero de acceso a base de datos, esto podría afectar el rendimiento

3.2 Compra con cliente inexistente

En este caso, Hibernate únicamente ejecuta una consulta select sobre la tabla cliente

Insertar compra con cliente inexistente

Hibernate:

```
select
    cliente0_.cif as cif1_1_0_,
    cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
    cliente0_.descripcion as descripcion2_1_0_,
    cliente0_.ciudad as ciudad3_1_0_,
    cliente0_.cp as cp4_1_0_,
    cliente0_.direccion as direccion5_1_0_,
    bonoclient1_.idbonocliente as idbonocliente1_0_1_,
    bonoclient1_.bono as bono2_0_1_,
    bonoclient1_.descuento as descuento3_0_1_
from
    HR.Cliente cliente0_
left outer join
    HR.BonoCliente bonoclient1_
        on cliente0_.IDBONOCLIENTE=bonoclient1_.idbonocliente
where
    cliente0_.cif=?
```

Esta consulta intenta recuperar el cliente junto con su posible bono asociado usando para ello un LEFT OUTER JOIN. Sin embargo, como no existe ningún cliente con el CIF indicado, la consulta no devuelve resultados.

Como consecuencia de esto, la operación se detiene en este punto y se lanza la excepción:

```
2026-05-04 20:42:03 ERROR ServiceImpl:235 - Error de transacción clasificado:
No existe cliente
```

3.3 Compra con menú inexistente

En este caso, Hibernate ejecuta dos consultas SELECT. En primer lugar, realiza una consulta sobre la tabla Cliente para comprobar su existencia y obtener la información asociada, luego ejecuta la segunda consulta sobre la tabla Menu, esta no devuelve resultados al no existir el menú indicado.

Como consecuencia, la operación se detiene en este punto y lanza la excepción correspondiente

2026-05-04 22:02:46 ERROR ServiceImpl:235 - Error de transacción clasificado:
No existe menú

3.4 Insertar compra con fecha y cliente existentes

En este caso, si se llegan a ejecutar las tres consultas SELECT, una sobre Cliente, otra sobre Menu y la tercera sobre Compra, que permite comprobar la existencia de una compra con el mismo CIF y fecha.

Al detectarse, que la compra ya existe, esta lanza la excepción correspondiente y no se ejecuta la operación INSERT

2026-05-04 22:11:17 ERROR ServiceImpl:235 - Error de transacción clasificado:
Compra ya existente

3.5 Quitar descuento del cliente

En primer lugar, Hibernate ejecuta la siguiente consulta:

Hibernate:

```
select
    cliente0_.cif as cif1_1_0_,
    cliente0_.IDBONOCIENTE as idbonocliente6_1_0_,
    cliente0_.descripcion as descripcion2_1_0_,
    cliente0_.ciudad as ciudad3_1_0_,
    cliente0_.cp as cp4_1_0_,
    cliente0_.direccion as direccion5_1_0_,
    bonoclient1_.idbonocliente as idbonocliente1_0_1_,
    bonoclient1_.bono as bono2_0_1_,
    bonoclient1_.descuento as descuento3_0_1_
from
    HR.Cliente cliente0_
left outer join
    HR.BonoCliente bonoclient1_
        on cliente0_.IDBONOCIENTE=bonoclient1_.idbonocliente
where
    cliente0_.cif=?
```

En esta consulta se recupera el cliente junto con su bono, esto permite obtener el descuento que debe eliminarse.

A continuación, se ejecuta:

Hibernate:

```
select
    compras0_.CIF as cif1_2_0_,
```

```

        compras0_.fecha as fecha2_2_0_,
        compras0_.cif as cif1_2_1_,
        compras0_.fecha as fecha2_2_1_,
        compras0_.importe as importe3_2_1_,
        compras0_.IDMENU as idmenu5_2_1_,
        compras0_.personas as personas4_2_1_,
        menu1_.idmenu as idmenu1_3_2_,
        menu1_.descripcion as descripcion2_3_2_,
        menu1_.precio as precio3_3_2_
from
    HR.Compra compras0_
left outer join
    HR.Menu menu1_
        on compras0_.IDMENU=menu1_.idmenu
where
    compras0_.CIF=?

```

Con esta consulta se obtienen todas las compras que están asociadas al cliente, junto con la información del menú necesaria para poder calcular el importe.

Luego, estas dos consultas Cliente y Compra se vuelven a ejecutar, ya que Hibernate vuelve a acceder a las entidades antes de sincronizar los cambios con la base de datos

Hibernate:

```

select
    cliente0_.cif as cif1_1_0_,
    cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
    cliente0_.descripcion as descripcion2_1_0_,
    cliente0_.ciudad as ciudad3_1_0_,
    cliente0_.cp as cp4_1_0_,
    cliente0_.direccion as direccion5_1_0_,
    bonoclient1_.idbonocliente as idbonocliente1_0_1_,
    bonoclient1_.bono as bono2_0_1_,
    bonoclient1_.descuento as descuento3_0_1_
from
    HR.Cliente cliente0_
left outer join
    HR.BonoCliente bonoclient1_
        on cliente0_.IDBONOCLIENTE=bonoclient1_.idbonocliente
where
    cliente0_.cif=?

```

Hibernate:

```

select
    compras0_.CIF as cif1_2_0_,
    compras0_.fecha as fecha2_2_0_,
    compras0_.cif as cif1_2_1_,
    compras0_.fecha as fecha2_2_1_,

```

```

        compras0_.importe as importe3_2_1_,
        compras0_.IDMENU as idmenu5_2_1_,
        compras0_.personas as personas4_2_1_,
        menu1_.idmenu as idmenu1_3_2_,
        menu1_.descripcion as descripcion2_3_2_,
        menu1_.precio as precio3_3_2_
from
    HR.Compra compras0_
left outer join
    HR.Menu menu1_
        on compras0_.IDMENU=menu1_.idmenu
where
    compras0_.CIF=?

```

Finalmente, se ejecutan tres UPDATES

Hibernate:

```

update HR.Compra
set importe=?, IDMENU=?, personas=?
where cif=? and fecha=?

```

Hibernate:

```

update HR.Compra
set importe=?, IDMENU=?, personas=?
where cif=? and fecha=?

```

Hibernate:

```

Update HR.Compra
set importe=?, IDMENU=?,
    personas=?
where cif=? and fecha=?

```

Estas operaciones UPDATES se realizan una vez por cada compra del cliente, actualizando de forma individual cada registro.

Este comportamiento evidencia que JPA no realiza actualizaciones en bloque, sino que sincroniza cada entidad, esto puede afectar el rendimiento en colecciones grandes.

3.6 Información completa con grafos de entidades

En primer lugar, se ejecuta la siguiente consulta

Hibernate:

```
select
    menu0_.idmenu as idmenu1_3_0_,
    menu0_.descripcion as descripcion2_3_0_,
    menu0_.precio as precio3_3_0_
from
    HR.Menu menu0_
where
    menu0_.idmenu=?
```

Esta consulta recupera el menú solicitado

A continuación, se ejecuta la siguiente consulta

Hibernate:

```
select
    menu0_.idmenu as idmenu1_3_0_,
    compras1_.cif as cif1_2_1_,
    compras1_.fecha as fecha2_2_1_,
    cliente2_.cif as cif1_1_2_,
    menu0_.descripcion as descripcion2_3_0_,
    menu0_.precio as precio3_3_0_,
    compras1_.importe as importe3_2_1_,
    compras1_.IDMENU as idmenu5_2_1_,
    compras1_.personas as personas4_2_1_,
    compras1_.IDMENU as idmenu5_2_0_,
    compras1_.cif as cif1_2_0_,
    compras1_.fecha as fecha2_2_0_,
    cliente2_.IDBONOCIENTE as idbonocliente6_1_2_,
    cliente2_.descripcion as descripcion2_1_2_,
    cliente2_.ciudad as ciudad3_1_2_,
    cliente2_.cp as cp4_1_2_,
    cliente2_.direccion as direccion5_1_2_
from
    HR.Menu menu0_
left outer join
    HR.Compra compras1_
        on menu0_.idmenu=compras1_.IDMENU
left outer join
    HR.Cliente cliente2_
        on compras1_.CIF=cliente2_.cif
```

En esta consulta se recupera el menú junto con sus compras y los clientes que están asociados mediante varios LEFT OUTER JOIN, esto permite obtener gran parte de la información relacionada en una sola operación.

Sin embargo, después se observan cuatro consultas adicionales sobre la tabla Compra. Estas consultas se ejecutan para recuperar la información asociada a cada cliente de forma individual.

Esto ocurre porque, aunque inicialmente se ha realizado una consulta con JOIN que obtiene parte de la información relacionada, Hibernate no carga completamente todas las relaciones en esa única consulta. En su lugar, accede de nuevo a la base de datos cuando necesita esa información adicional.

Esto se debe a que la relación entre Menu y Compra esta definida como LAZY (por defecto en la anotación @OneToMany). Como consecuencia, las compras no se cargan completamente en la consulta inicial, generándose consultas adicionales al acceder a dicha operación.

Este patrón puede dar lugar al problema conocido como N+1 consultas

Hibernate:

```
select
    compras0_.CIF as cif1_2_0_,
    compras0_.fecha as fecha2_2_0_,
    compras0_.cif as cif1_2_1_,
    compras0_.fecha as fecha2_2_1_,
    compras0_.importe as importe3_2_1_,
    compras0_.IDMENU as idmenu5_2_1_,
    compras0_.personas as personas4_2_1_,
    menu1_.idmenu as idmenu1_3_2_,
    menu1_.descripcion as descripcion2_3_2_,
    menu1_.precio as precio3_3_2_
from
    HR.Compra compras0_
left outer join
    HR.Menu menu1_
        on compras0_.IDMENU=menu1_.idmenu
where
    compras0_.CIF=?
```

Hibernate:

```
select
    compras0_.CIF as cif1_2_0_,
    compras0_.fecha as fecha2_2_0_,
    compras0_.cif as cif1_2_1_,
    compras0_.fecha as fecha2_2_1_,
    compras0_.importe as importe3_2_1_,
    compras0_.IDMENU as idmenu5_2_1_,
    compras0_.personas as personas4_2_1_,
    menu1_.idmenu as idmenu1_3_2_,
    menu1_.descripcion as descripcion2_3_2_,
    menu1_.precio as precio3_3_2_
```

```

from
    HR.Compra compras0_
left outer join
    HR.Menu menu1_
        on compras0_.IDMENU=menu1_.idmenu
where
    compras0_.CIF=?
Hibernate:
select
    compras0_.CIF as cif1_2_0_,
    compras0_.fecha as fecha2_2_0_,
    compras0_.cif as cif1_2_1_,
    compras0_.fecha as fecha2_2_1_,
    compras0_.importe as importe3_2_1_,
    compras0_.IDMENU as idmenu5_2_1_,
    compras0_.personas as personas4_2_1_,
    menu1_.idmenu as idmenu1_3_2_,
    menu1_.descripcion as descripcion2_3_2_,
    menu1_.precio as precio3_3_2_
from
    HR.Compra compras0_
left outer join
    HR.Menu menu1_
        on compras0_.IDMENU=menu1_.idmenu
where
    compras0_.CIF=?
Hibernate:
select
    compras0_.CIF as cif1_2_0_,
    compras0_.fecha as fecha2_2_0_,
    compras0_.cif as cif1_2_1_,
    compras0_.fecha as fecha2_2_1_,
    compras0_.importe as importe3_2_1_,
    compras0_.IDMENU as idmenu5_2_1_,
    compras0_.personas as personas4_2_1_,
    menu1_.idmenu as idmenu1_3_2_,
    menu1_.descripcion as descripcion2_3_2_,
    menu1_.precio as precio3_3_2_
from
    HR.Compra compras0_
left outer join
    HR.Menu menu1_
        on compras0_.IDMENU=menu1_.idmenu
where
    compras0_.CIF=?

```

4 Reflexión personal

Esta práctica me ha permitido comprender mejor el funcionamiento interno de JPA y todo lo que implica el uso de un framework de persistencia. A partir de unas pocas líneas de código, se generan muchas operaciones SQL, lo que pone de manifiesto la complejidad que existe detrás de la abstracción ofrecida por estas herramientas.

Además, considero que este tipo de prácticas resulta especialmente útil para entender la base sobre la que se apoyan frameworks más avanzados como Spring Boot, ampliamente utilizados en entornos reales.

Comprender como se gestionan las entidades, las relaciones y las consultas permiten tener una visión mas completa de como funcionan estos sistemas en producción.

En general, me ha parecido una practica muy provechosa para afianzar los conocimientos adquiridos.

5 Conclusión

El análisis de las operaciones SQL generadas por Hibernada ha permitido identificar como se traduce cada transacción de alto nivel en múltiples consultas sobre la base de datos. A través de este estudio, se ha podido observar el impacto de la gestión de entidades, las relaciones y las estrategias de carga en el numero y tipo de operaciones ejecutadas. Este conocimiento resulta fundamental para poder utilizar JPA de forma eficiente, evitando un uso ineficiente de la base de datos y mejorando el rendimiento de las aplicaciones.